

Manikstu CRM

Project Workflow — how the system works, end to end

Manikstu runs **two independent systems**. The public website takes orders and contact-form enquiries from farmers. The **CRM** is the internal cockpit where staff turn those leads and enquiries into sales and fulfil orders. They never share code or a database.

1. Two separate systems

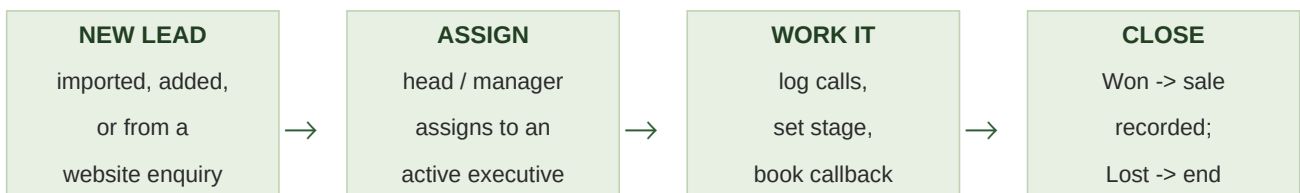
	Public website	CRM (what we built / wired)
Frontend	marketing / e-commerce site	manikstu_crm (Next.js dashboards)
Backend	manikstu-backend (untouched)	manikstu_crm_backend (Laravel + PostgreSQL)
Users	farmers / public	Manikstu staff

2. The four CRM roles (one app, four dashboards)

Manager	The big picture: sales pipeline, orders, farmers, regional report, inventory, finance and reports, plus a read-only view of the telecalling team. Can reassign leads and act on enquiries.
Telecalling head	Runs the calling team: imports and assigns leads, onboards staff (creates Staff IDs + temporary passwords), handles enquiries and complaints.
Calling executive	Works only their own assigned leads: makes calls, logs outcomes, books follow-ups, records sales.
Seller	Sees only their own products and the orders that contain them.

3. The core workflow — a lead's life

This shared "tracker" is the heart of the system; it connects the head, the executives and the manager in real time.



Rules the backend enforces (ported exactly from the original app):

- A lead belongs to one **vertical** — Goat Health Products, Goat Insurance, or Goat Bank — each with its own **stages** (e.g. Health: New → Contacted → Interested → Sample sent → Won / Lost).
- **Assign** only to an active executive; open follow-ups move with the lead.
- Each **call** has an outcome (Connected / No answer / Busy / Wrong number), optional duration and note, and can advance the stage. A **Connected** call auto-closes the due callback and can book the next one.
- Reaching a **Won** stage records a **sale**; physical-product sales also become telecaller **orders**.
- A **calling executive** sees only their own leads; the head and manager see everyone.
- The manager and head also triage website **enquiries** (new → read → replied → archived) and can link an enquiry to a lead.

4. How the pieces talk (request flow)

Browser (dashboard)

Client components call the app's own `/api/*` routes — no CORS, and the token stays server-side.



Next.js route / server page

Reads `httpOnly` cookies and attaches the Sanctum token (`mk_api`) before calling the backend.



Laravel backend `/api/*`

`TrackerService` and controllers apply the rules and talk to the database.



PostgreSQL

Returns JSON, already scoped to the caller's role.

Live sync: every dashboard polls `GET /api/tracker?since=<version>` every 5 seconds. The backend keeps a monotonic version; if nothing changed it replies "unchanged", otherwise it returns fresh state — so when an executive logs a call, the manager's team view updates on its own.

Initial render: each role page (`/manager`, `/telecaller`, ...) fetches its starting data from the backend on the server and renders it.

5. Authentication flow

- Sign in with **email**, **Staff ID or mobile** + password.
- The login route calls the backend, which verifies and returns a **Sanctum token**, the user, and a **mustChangePassword** flag.
- The app sets two `httpOnly` cookies: **mk_session** (signed — drives middleware and server-side role-gating) and **mk_api** (the token used for data calls).
- If **mustChangePassword** is set, the user is sent to **/change-password** before anything else; otherwise straight to their role dashboard.
- Edge **middleware** checks `mk_session` on every page and redirects to the right dashboard or to `/login`. Sessions last one 8-hour shift; logout revokes the token and clears both cookies.

6. What the backend owns (PostgreSQL)

Identity & staff	users with Staff IDs and onboarding fields; Sanctum tokens.
Tracker	verticals, stages, products, leads, <code>lead_activities</code> , follow-ups, sales, + a version counter.
Commerce	orders (website + telecaller) with items and status history; customers (farmers).
Enquiries	website contact-form enquiries, linkable to a lead.
Inventory	catalogue products, stock points (warehouses + hubs), stock per location.
Finance & partners	transactions / settlements, franchises, FPOs.
Complaints	tickets with status and assignee.

Role gating mirrors the lead-action matrix: only the head / manager assign leads; only executives log calls; sellers cannot see the tracker.

7. Running it locally

Start the backend, then the frontend:

```
cd manikstu_crm_backend && php artisan serve --port=8000
cd manikstu_crm && npm run dev (opens http://localhost:3000)
```

Demo logins

Role	Login	Password
Manager	manager@manikstu.com / MNK-MGR-042	manager123
Telecalling head	pradeep@manikstu.in / MK-TL-101	telecaller123
Calling executive	bikash@manikstu.in / MK-TC-012	executive123
Seller	sanjay@manikstu.in / MK-SL-301	seller123

The loop in one line: website / enquiry → lead → assigned by head or manager → worked by a calling executive → won into a sale / order → fulfilled and settled → rolled up for the manager.